A Survey of Exact Algorithms for Subset Sum and Its Variants

Le Duc Minh
minh.leduc.0210@gmail.com

October 2026

Abstract

Subset sum asks whether some of n given integers add up to a target. For every input, the best known exact algorithm still takes about $2^{n/2}$ steps, as it did in 1974. A great deal has happened around that bound. On random inputs, algorithms based on the representation technique run in about $2^{0.283n}$ steps. When the numbers are small, algorithms that are nearly linear in the target are known. For several close relatives of subset sum, worst-case bounds below $2^{n/2}$ have been proved in the last few years, including $2^{n/3}$ for the pigeonhole version of equal subset sum. This survey organises these results by the question each one answers: how fast on every input, how fast on random inputs, and how fast when the numbers are small. For each line of work we explain the main idea, compare the bounds, and say what the idea needs from the input. We then use these comparisons to explain why plain subset sum has resisted so far, collect the known lower bounds, and list open problems.

Keywords: subset sum, knapsack, equal subset sum, exact exponential algorithms, meet-in-the-middle, representation technique, survey

1 Introduction

1.1 The problem

Given integers $a_1, \ldots, a_n$ and a target b, *subset sum* asks whether some of the a_i add up exactly to b; in vector form, whether some $x \in \{0,1\}^n$ satisfies $\sum_i a_i x_i = b$. The search version also asks for such an x. For example, $3, 5, 8, 13$ reach 16 as $3+13$ and as $3+5+8$, but never reach 2. Subset sum is the yes/no form of the 0–1 knapsack problem and one of the standard NP-complete problems. It also has a long history in cryptography: the Merkle–Hellman cryptosystem [1] based its security on it, and the algorithms surveyed here are still used to analyse attacks on modern schemes.

1.2 The central question

Since NP-complete problems are not expected to have polynomial algorithms, the question for subset sum is how small the exponential part can be. Trying all subsets costs 2^n steps. In 1974 Horowitz and Sahni [7] brought this down to about $2^{n/2}$, and no algorithm known today beats $2^{(1/2-\varepsilon)n}$ on every input for any constant $\varepsilon > 0$. Whether such an algorithm exists, and in particular whether $2^{n/3}$ is possible, is one of the best-known open problems in exact algorithms. The constant matters in practice: for $n = 100$, $2^{n/2}$ is about 10^{15} while $2^{n/3}$ is about 10^{10}.

Table 1: Three ways to measure progress, and the best known classical bound in each.

Setting	Best known	Where it is hard
Worst case in n	$2^{n/2}/n^{0.5023}$ [9]	numbers of about n bits
Average case	$2^{0.283n}$, heuristic [16]	density close to 1
Pseudopolynomial	$\tilde{\mathcal{O}}(n+t)$ [36]	large target t

1.3 Three ways to measure progress

Results on subset sum are stated in three different ways, and much of the apparent progress on "beating $2^{n/2}$" depends on which one is meant (Table 1).

- *Worst case in n.* A bound in terms of n that holds for every input. This is the setting of the central question.
- *Average case.* A bound for inputs drawn at random, usually of density about 1 (numbers of about n bits), often under an extra unproved assumption about how random inputs behave.
- *Pseudopolynomial.* A bound in terms of n and the target t (or the largest number), which is fast when the numbers are small.

An average-case or pseudopolynomial bound below $2^{n/2}$ says nothing about the worst case, because the hard inputs may be exactly the ones the bound excludes.

1.4 Scope and organisation

We cover exact algorithms for subset sum and its closest relatives, both classical and quantum. We do not cover approximation schemes, the optimisation version of knapsack, or parallel algorithms. Section 2 fixes notation and the variants. Sections 3–6 cover the worst-case, average-case and promise results in turn. Section 7 covers small numbers, Section 8 random inputs at other densities, and Section 9 lower bounds. Section 10 compares all of these and draws out common themes, and Section 11 lists open problems.

2 Problems and Notation

2.1 Measures

We write n for the number of integers, t for the target, and $W = \sum_i |a_i|$. The *density* of an input is $\delta = n/\log_2 \max_i |a_i|$; density 1 means the numbers have about n bits. We write $\mathcal{O}^*(f)$ for f times a polynomial in n and the input length, and $\tilde{\mathcal{O}}(f)$ for f times a polylogarithmic factor.

2.2 One problem, many coefficient sets

Most variants fit one template. Fix a finite set C of integers, the *coefficient set*, and ask for $c \in C^n$ with $\sum_i c_i a_i$ equal to the target, with $c \neq 0$ when the target is 0. Randolph and Węgrzycki [17] call this *subset balancing*. Subset sum is $C = \{0, 1\}$. Partition, which splits the numbers into two groups of equal sum, is $C = \{-1, 1\}$ with target 0. Equal subset sum (ESS), which asks for two different subsets with the same sum, is $C = \{-1, 0, 1\}$ with target 0. We write $[\pm d] = \{-d, \ldots, -1, 1, \ldots, d\}$ and $[-d : d] = \{-d, \ldots, d\}$. Table 2 lists the variants used in this survey.

Table 2: Problem variants, coefficient sets, and promises.

Variant	Coefficient set / promise
Subset Sum	$C = \{0, 1\}$, target b
Partition	$C = \{-1, 1\}$, target 0
Equal Subset Sum (ESS)	$C = \{-1, 0, 1\}$, target 0
Pigeonhole ESS (PESS)	ESS on positive a_i with $\sum_i a_i < 2^n - 1$ (solution guaranteed)
Subset Balancing	fixed C, e.g. $[\pm d]$ or $[-d : d]$
k-SUM	one element from each of k lists, sum to target
Fixed weight	$C = \{0, 1\}$ with exactly k items chosen
Modular	the equation holds modulo some M

2.3 How the variants relate

Partition and subset sum are equally hard up to polynomial factors: Partition is subset sum with target $W/2$, and a subset sum instance becomes a Partition instance after adding two suitable numbers. ESS looks for a *collision* between two subset sums rather than a specific value, so its trivial bound is $3^{n/2}$ (meet-in-the-middle over $\{-1, 0, 1\}$) rather than $2^{n/2}$. PESS adds the promise $\sum_i a_i < 2^n - 1$. Then 2^n subsets share fewer than $2^n - 1$ possible sums, so by the pigeonhole principle two of them must have the same sum, and the problem is to find them. Papadimitriou [2] introduced PESS as a natural problem in the complexity class PPP of such pigeonhole searches.

3 Worst-Case Algorithms

This section follows the main line of worst-case results in n. Its story is short: the time bound $2^{n/2}$ has stood since 1974, and most later progress has been about memory or polynomial factors.

3.1 Meet-in-the-middle

Horowitz and Sahni [7] split the numbers into a left and a right half, list the $2^{n/2}$ subset sums of each half, sort one list, and for each left sum u search the other for $b - u$. A solution exists exactly when some pair matches, because every subset splits uniquely into a left part and a right part. This takes $\mathcal{O}^*(2^{n/2})$ time and memory, is deterministic, and returns a witness. Every later worst-case algorithm in this section is a refinement of this idea.

3.2 Saving memory

The $2^{n/2}$ stored sums are often the practical bottleneck. Schroeppel and Shamir [8] split the input into four quarters instead of two halves. They sort the four quarter lists, of $2^{n/4}$ entries each, and use two priority queues to produce the sums of the first two quarters in increasing order and the sums of the last two quarters in decreasing order. A two-pointer sweep over these streams finds a match without ever storing the half lists. The time stays $\mathcal{O}^*(2^{n/2})$, and the memory drops to $\mathcal{O}^*(2^{n/4})$. More generally, with memory $S \le 2^{n/4}$ one can get time about $2^n/S^2$.

For about forty years $2^{n/4}$ memory was the best known at time $2^{n/2}$. Nederlof and Węgrzycki [10] broke this barrier with a randomized algorithm that uses memory $\mathcal{O}^*(2^{0.249999n})$, by relating subset sum to the Orthogonal Vectors problem. Belova, Chukhin, Kulikov and Mihajlin [11] lowered the memory exponent to 0.246. These results do not change the time.

At the other extreme, one can ask for memory polynomial in n. Lokshtanov and Nederlof [13] gave pseudopolynomial algorithms in polynomial space, and Bansal, Garg, Nederlof and Vyas [12] solved subset sum in $\mathcal{O}^*(2^{0.86n})$ time with polynomial memory.

3.3 Shaving polynomial factors

The first time improvement over Horowitz and Sahni for every input came almost fifty years later. Chen, Jin, Randolph and Servedio [9] solved subset sum in $\mathcal{O}(2^{n/2}/n^{\gamma})$ time for a constant $\gamma > 0.5023$, in the standard word-RAM model. Their algorithm combines representation-style ideas with bit-packing, so that many comparisons fit into one machine word. The exponent $1/2$ is unchanged.

3.4 Summary

For every input, the time exponent of subset sum has been $1/2$ for fifty years. Memory has improved from $2^{n/2}$ to below $2^{n/4}$, and the time has improved by a polynomial factor. The ideas that do lower the exponent come from the average-case setting, which we turn to next.

4 Random Inputs: The Representation Technique

4.1 The key idea

Meet-in-the-middle fixes one split of the *positions* and needs a solution to split along it. Howgrave-Graham and Joux [14] instead split the *solution*. A solution x that picks ℓ numbers can be written as $x = y + z$, where y and z each pick $\ell/2$ of those numbers. There are $\binom{\ell}{\ell/2} \approx 2^{\ell}$ ways to do this, and any one of them is enough to find x. So the algorithm can afford to throw most of them away. It picks a modulus M close to the number of splits and a random residue R, and keeps only the vectors y whose sum is R modulo M. If the residues of the splits are spread evenly, about one split survives, while the list of candidates shrinks by a factor of M.

The ideal two-way version gives lists of size about $2^{0.3113n}$ for a solution of weight $n/2$. The difficulty is to build these filtered lists in time close to their size, rather than by listing all candidates first.

4.2 From 0.337 to 0.283

Howgrave-Graham and Joux built the lists by applying the same idea one level down: they split the solution into four pieces, match pairs of pieces under one modulus, and then match the two pair sums exactly. Merging two lists of size L under a modulus M produces about L^2/M pairs, and this merge cost sets the final time. With the top modulus $2^{\beta n}$, the time is $\max(2^{0.3113n}, 2^{(0.0872+\beta)n})$ and the memory is $2^{(0.5436-\beta)n}$ for $1/4 < \beta < 1/3$. Table 3 shows the resulting trade-off; the fastest setting gives time $2^{0.337n}$.

Becker, Coron and Joux [15] made the pieces more flexible. They allow the coefficient -1 inside the pieces, so that an unchosen number can be split as $0 = 1 + (-1)$ as well as $0 + 0$. This gives many more splits per solution, which allows stronger filters and smaller lists. With three levels of splitting, their time is $2^{0.291n}$. Bonnetain, Bricout, Schrottenloher and Shen [16] allowed four coefficient values and reached $2^{0.283n}$, the best classical bound for random inputs that we know of.

Table 3: Time–memory trade-off of the four-way algorithm of [14], as exponents of 2^n.

β	time	memory
$\to 1/4$	0.337	0.294
0.2936	0.381	0.250
0.2972	0.385	0.247
$\to 1/3$	0.421	0.211

4.3 The price of flexibility

More flexible pieces also create *pseudo-solutions*: pairs of pieces that pass every filter and add up to the target, but whose sum is not a valid 0/1 vector, for instance because it has a 2 or a -1 in some position. The algorithm has to reject these, and if there are too many the lists grow again. Later worst-case work handles this by checking that a pair is *compatible* on a small random sample of positions before combining it [17].

4.4 Why these bounds are average-case

All of these analyses assume that the residues of a solution's splits are spread evenly modulo M, and that the filtered lists have their expected sizes. For random numbers of about n bits this is very plausible, and experiments agree. For a chosen input it can fail: if every number is a multiple of M, every sum has residue 0 and the filter removes nothing. Turning the technique into a worst-case algorithm means handling such inputs, which is the subject of the next section.

5 From Random to Worst Case

5.1 Mixing and structure

Randolph and Węgrzycki [17] recently made the representation technique work on every input for many coefficient sets. Their main tool is a *dichotomy*. Either the splits of a solution are spread well modulo the filter, in which case the filter works as in the random case, or they are not, in which case the input has special additive structure. Examples of such structure are a small set of distinct subset sums, or exponentially many solutions. That structure can then be used by a different algorithm, such as subsampling or a dense dynamic program. Either way the input is solved faster than $2^{n/2}$.

5.2 Coefficient shifting

The second ingredient creates extra splits when the coefficient set does not provide them directly. They write the coefficient set as a *sumset* $C = C_1 + C_2 = \{c_1 + c_2 : c_1 \in C_1, c_2 \in C_2\}$ and split each coefficient vector into one part over C_1 and one over C_2. This gives many splits only if some coefficient has at least two representations $c_1 + c_2$. For example, $\{0, 1\} + \{-3, -2, 1, 2\} = [\pm 3]$, where $-2 = 0 + (-2) = 1 + (-3)$ and $2 = 0 + 2 = 1 + 1$.

5.3 What is solved and what is open

With these tools they beat $|C|^{n/2}$ in the worst case for $[-d : d]$ with $d \geq 2$, for $[\pm d]$ with $d \geq 3$, and for ESS, which they solve in 1.7067^n time (Table 4). Three coefficient sets remain open: $\{0, 1\}$ (subset sum), $\{-1, 1\}$ (Partition) and $[\pm 2] = \{-2, -1, 1, 2\}$. None of the three can be written as

Table 4: Worst-case subset balancing over coefficient sets C [17]. The baseline is meet-in-the-middle, $|C|^{n/2}$.

C	Worst-case result
$[-d:d]$, $d \geq 2$	$\lvert C\rvert^{(1/2-\epsilon)n}$ for some $\epsilon > 0$
$[\pm d]$, $d \geq 3$	$\lvert C\rvert^{(1/2-\epsilon)n}$ for some $\epsilon > 0$
$\{-1, 0, 1\}$ (ESS)	1.7067^n (baseline 1.7321^n)
$\{-1, 1\}$ (Partition)	open
$[\pm 2]$	open
$\{0, 1\}$ (Subset Sum)	open

$C_1 + C_2$ so that some coefficient has two representations. For the two-element sets this follows from $|C_1 + C_2| \geq |C_1| + |C_2| - 1$. For $[\pm 2]$ it follows from the fact that equality in that inequality forces an arithmetic progression, which $[\pm 2]$ is not. So coefficient shifting has nothing to work with on exactly the cases that remain open. This suggests that a worst-case algorithm for subset sum below $2^{n/2}$ needs a different source of extra splits.

6 Where $2^{n/2}$ Has Been Beaten

Several close relatives of subset sum now have worst-case algorithms below their meet-in-the-middle baseline. Each one has some feature that plain subset sum lacks, and comparing these features shows what a breakthrough for subset sum might need.

6.1 Pigeonhole equal subset sum

In PESS the numbers are positive and add up to less than $2^n - 1$. There are 2^n subsets but fewer than $2^n - 1$ possible sums, so two subsets must share a sum. A solution always exists, and the task is to find one.

A simple $2^{n/2}$ algorithm. The pigeonhole argument can be turned into a binary search. Call an interval $[\ell, r]$ *crowded* if more than $r - \ell + 1$ subsets have their sum in it. A crowded interval must contain two subsets with the same sum. The whole range $[0, 2^n - 2]$ is crowded. Split a crowded interval in half: at least one half is crowded, because otherwise the two halves would together hold too few subsets. After about n halvings the interval is a single crowded value, and two subsets with that sum can be recovered. Each step needs the number of subsets with sum at most some value T, which meet-in-the-middle computes in $\mathcal{O}^*(2^{n/2})$ time, so the whole search takes $\mathcal{O}^*(2^{n/2})$. Algorithm 1 summarises it.

Breaking $2^{n/2}$. Jin and Wu [20] gave the first faster algorithm, in $\mathcal{O}^*(2^{0.4n})$ time. Their key quantity is the number d of values in $[0, 2^n)$ that are *not* subset sums. This equals the total number of "extra" subsets that collide with an earlier one, so it measures how many solutions there are. They split into two cases.

- If d is small, almost every value is a subset sum. They show that the sorted numbers must then be very close to $1, 2, 4, 8, \ldots$: each a_i differs from 2^{i-1} by at most a small multiple of d. For such numbers, the count of subsets with sum at most T can be computed using meet-in-the-middle on only a short prefix, so each binary-search step becomes much cheaper.

Algorithm 1 Pigeonhole binary search for PESS

Require: positive integers $a_1, \ldots, a_n$ with $\sum_i a_i < 2^n - 1$
Ensure: two different subsets with equal sum

```
ℓ ← 0; r ← Σ_i a_i
while ℓ < r do
    m ← ⌊(ℓ + r)/2⌋
    if #{S : ℓ ≤ w(S) ≤ m} > m − ℓ + 1 then        ▷ count by meet-in-the-middle
        r ← m
    else
        ℓ ← m + 1
    end if
end while
return two subsets with sum ℓ
```

- If d is large, there are many colliding pairs. They pick a random prime p and a random residue class modulo p, sample subsets from that class, and look for two with equal sum. Many collisions make this likely to succeed quickly.

Balancing the two cases gives $2^{0.4n}$. Jin, Williams and Zhang [29] then improved the bound to $\mathcal{O}^*(2^{n/3})$. Jin and Wu also gave an $\mathcal{O}^*(2^{0.75n})$ algorithm that uses only polynomial memory.

PESS is the clearest example of a relative of subset sum that reaches $2^{n/3}$ in the worst case. Its promise does two things that plain subset sum lacks. It guarantees a solution, and it forces either many solutions or a very rigid structure, and both cases can be used.

6.2 Equal subset sum

ESS asks for two different subsets with the same sum, with no promise. Its meet-in-the-middle baseline is $3^{n/2} \approx 1.7321^n$, since each number can go to the first subset, the second, or neither. Mucha, Nederlof, Pawlewicz and Węgrzycki [22] were the first to beat it, with 1.7088^n, using a worst-case form of the representation technique. The key point is that ESS has the coefficient 0 and the target 0, so a solution has many representations for free: $0 = 0 + 0 = 1 + (-1) = (-1) + 1$. Randolph and Węgrzycki [17] improved this to 1.7067^n. Two 2026 preprints report further improvements by branching on the zero-density of a minimum-support solution: Yamano and Shibuya [18] give 1.6994^n time and 1.5664^n space, and Ye [19] claims a one-sided Monte Carlo bound of $(5/3)^n \approx 1.6667^n$.

6.3 Either-or subset sum

Randolph [30] studied a hybrid problem: given an input, return *either* a subset with the target sum *or* two different subsets with equal sum. He showed that this can be solved in randomized $\mathcal{O}^*(2^{0.461n})$ time on every input. This has a direct consequence for subset sum itself. On an input whose 2^n subset sums are all different, there is no equal-sum pair to return, so the algorithm must find a subset with the target sum whenever one exists. Subset sum is therefore solvable below $2^{n/2}$ on all such inputs. This is the closest known worst-case result to subset sum. It shows that any inputs that block a faster subset sum algorithm must have some colliding subset sums.

6.4 Quantum algorithms

On a quantum computer, collision-finding methods based on Grover search solve subset sum in $\tilde{\mathcal{O}}(2^{n/3})$ time on every input. Bernstein, Jeffery, Lange and Meurer [31] combined quantum search with the representation technique and obtained a heuristic average-case bound of $2^{0.241n}$. Chukhin, Kulikov, Levitskii and Mihajlin [32] recently gave a worst-case quantum bound of $\tilde{\mathcal{O}}(2^{2n/7}) \approx 2^{0.286n}$ through a worst-case quantum k-SUM algorithm. So the exponent $1/3$ is not a natural limit in general; the classical gap is a matter of algorithms we have not found yet.

6.5 Fixed weight and k-SUM

If the solution must contain exactly k numbers, the problem becomes k-SUM: choose k numbers that add up to the target. For constant k, meet-in-the-middle solves it in about $n^{\lceil k/2 \rceil}$ time, and the k-SUM conjecture states that this is essentially optimal. When k is a constant fraction of n, the representation technique applies as before. Under the extra constraint of polynomial memory, Esser and May [23] gave a heuristic $2^{0.65n}$ algorithm for random subset sum, improving the earlier polynomial-memory bound of about $2^{0.72n}$.

6.6 What these variants have in common

Each relative that beats its baseline in the worst case has something extra to work with. PESS has a guaranteed collision and a rigid structure when collisions are rare. ESS has the coefficient 0 and many free representations. Either-or subset sum may return a collision whenever one exists. The quantum algorithms can search faster. Plain subset sum has none of these: it may have a single solution, there is no promise, and its coefficient set $\{0, 1\}$ has no useful factorisation.

7 Small Numbers: Pseudopolynomial Algorithms

7.1 From Bellman to near-linear time

When the target t is small, the reachable sums fit in a table. Bellman's dynamic program from 1957 fills a table of the sums $0, \ldots, t$ one number at a time, in $\mathcal{O}(nt)$ steps. Packing table cells into machine words divides this by the word size. Pisinger [3] showed that $\mathcal{O}(n\, a_{\max})$ time is possible, where $a_{\max}$ is the largest number, which is better when the numbers are small compared to the target.

A newer line of work uses fast polynomial multiplication. The set of subset sums of a list is encoded as a polynomial whose exponents are the reachable sums. Combining two parts of the input then corresponds to multiplying their polynomials, which takes $\tilde{\mathcal{O}}(t)$ time with the fast Fourier transform. For example, the sums of $\{2\}$ and of $\{3\}$ are encoded by $1 + x^2$ and $1 + x^3$, and their product $1 + x^2 + x^3 + x^5$ encodes the sums $0, 2, 3, 5$ of $\{2, 3\}$. Koiliaris and Xu [35] used this to reach $\tilde{\mathcal{O}}(t\sqrt{n})$ deterministically. Bringmann [36] reached $\tilde{\mathcal{O}}(n + t)$ with a randomized algorithm that splits the numbers into groups by size and uses colour coding, and Jin and Wu [21] gave a simpler $\tilde{\mathcal{O}}(n + t)$ algorithm based on formal power series. Under the Strong Exponential Time Hypothesis, $t^{1-\epsilon}$ is not possible [28], so these algorithms are close to optimal in t.

7.2 Faster than Bellman in every setting

Later work refined the bound by other measures of the input. Table 5 lists the main results. Bringmann and Nakos [24] gave an *output-sensitive* algorithm, whose time depends on the number

Table 5: Pseudopolynomial algorithms for subset sum. Here t is the target, $a_{\max}$ the largest number and $|S(X,t)|$ the number of distinct subset sums up to t.

Algorithm	Time	Notes
Bellman (1957)	$\mathcal{O}(nt)$	table of reachable sums
Pisinger [3]	$\mathcal{O}(n\,a_{\max})$	good for small numbers
Koiliaris–Xu [35]	$\tilde{\mathcal{O}}(t\sqrt{n})$	deterministic, FFT
Bringmann [36]	$\tilde{\mathcal{O}}(n+t)$	randomized, colour coding
Jin–Wu [21]	$\tilde{\mathcal{O}}(n+t)$	randomized, power series
Bringmann–Nakos [24]	$\tilde{\mathcal{O}}(\lvert S(X,t)\rvert^{4/3})$	output-sensitive
Chen–Lian–Mao–Zhang [37]	$\tilde{\mathcal{O}}(n+\sqrt{a_{\max}t})$	
Bringmann–Fischer–Nakos [38]	$\tilde{\mathcal{O}}(\lvert S(X,t)\rvert\sqrt{n})$	beats Bellman everywhere

$|S(X,t)|$ of distinct reachable sums up to t. Chen, Lian, Mao and Zhang [37] gave $\tilde{\mathcal{O}}(n+\sqrt{a_{\max}t})$. Bringmann, Fischer and Nakos [38] gave $\tilde{\mathcal{O}}(|S(X,t)|\sqrt{n})$, which beats Bellman's algorithm for every combination of parameters.

7.3 Dense inputs and additive combinatorics

When there are many numbers compared to their size, a different phenomenon takes over. Results in additive combinatorics, such as those of Szemerédi and Vu [4], show that the subset sums of a large enough set of integers contain long arithmetic progressions. Once such a progression is found, every target inside a wide range can be reached by adjusting a few choices. Bringmann and Wellnitz [39] used this to decide dense instances in near-linear time, and Chen, Mao and Zhang [40] proved stronger structure theorems that also give a witness in near-linear time.

7.4 What this means for the worst case

Any of these algorithms beats $2^{n/3}$ when $t \le 2^{n/3}$. But the hard inputs for the central question have numbers of about n bits, so t can be close to 2^n, and pseudopolynomial bounds are then worse than meet-in-the-middle. The dense results point the same way from the other side. Inputs with many numbers relative to their size have structure that forces many subset sums, and that structure makes them easy. Hardness lives where the numbers are large and the subset sums are spread out.

8 Random Inputs at Other Densities

The representation technique of Section 4 targets density about 1. Random inputs of much lower or much higher density are easier, but for different reasons and with different tools.

8.1 Low density: lattices

When the numbers have many more than n bits, solutions are rare, and the unique solution can often be found as a short vector in a lattice built from the input. Lagarias and Odlyzko [34] showed that almost all random instances of density below 0.6463 can be solved in polynomial time given an oracle for the shortest vector problem. Coster et al. [27] raised the threshold to 0.9408. In practice,

Table 6: Random subset sum by density $\delta = n/\log_2 \max_i a_i$.

Density	Typical method	Behaviour
$\delta < 0.94$	lattice reduction [34, 27]	unique solution found as a short vector
$\delta \approx 1$	representation technique [15, 16]	about $2^{0.283n}$, the hardest case
$\delta \gg 1$	dynamic programming, generalised birthday [33]	many solutions, small numbers

Table 7: Conditional lower bounds and their assumptions.

Bound	Assumption
No $2^{o(n)}$ algorithm	Exponential Time Hypothesis [6]
No $t^{1-\epsilon}2^{o(n)}$ algorithm	Strong Exponential Time Hypothesis [28]
No $n^{\lceil k/2 \rceil - \epsilon}$ algorithm for k-SUM	k-SUM conjecture
No $2^{(1/2-\epsilon)n}$ algorithm for subset sum	meet-in-the-middle conjecture (unproved)

lattice reduction algorithms such as LLL [5] and its stronger variants approximate this oracle well in low dimension. These results are average-case, and they say nothing about density 1 or about the worst case.

8.2 High density: generalised birthday

When the numbers are small compared to 2^n but the problem is still posed modulo a large number, there are many solutions, and the generalised birthday algorithm of Wagner [33] applies. It splits the input into k lists and merges them in a tree, at each level keeping only pairs whose sum is zero in some low-order bits. For $k = 4$ this takes about $2^{n/3}$ time and memory, and for general k about $2^{n/(1+\log_2 k)}$. Lyubashevsky [25] and Shallue [26] applied this to random modular subset sum with a small modulus and obtained subexponential bounds. These algorithms need many solutions, so they do not apply to worst-case inputs with one solution.

8.3 The density picture

Table 6 puts the three regimes side by side. The hardest random inputs sit at density close to 1. That is also where the worst-case question is decided, because inputs of other densities can be handled by the methods above.

9 Lower Bounds and Barriers

No unconditional exponential lower bound is known for subset sum, since such a bound would separate P from NP. The known lower bounds are conditional: they hold if a widely believed hypothesis holds. Table 7 lists them.

The Exponential Time Hypothesis [6] implies that subset sum has no $2^{o(n)}$ algorithm, so some exponential dependence on n is necessary. The Strong Exponential Time Hypothesis implies that the pseudopolynomial bound $\tilde{\mathcal{O}}(n+t)$ cannot be improved to $t^{1-\epsilon}2^{o(n)}$ [28]. Neither hypothesis says anything about the constant in the exponent between 0 and 1/2. The k-SUM conjecture concerns a fixed number of chosen items and does not transfer to the setting where about $n/2$ items are chosen.

Table 8: Main results. WC: worst case; avg: average case, often heuristic; promise: holds under the PESS promise.

Result	Time	Memory	Setting
Horowitz–Sahni [7]	$2^{n/2}$	$2^{n/2}$	WC
Schroeppel–Shamir [8]	$2^{n/2}$	$2^{n/4}$	WC
Nederlof–Węgrzycki [10]	$2^{n/2}$	$2^{0.249999n}$	WC, randomized
Belova et al. [11]	$2^{n/2}$	$2^{0.246n}$	WC, randomized
Bansal et al. [12]	$2^{0.86n}$	polynomial	WC
Chen et al. [9]	$2^{n/2}/n^{0.5023}$		WC, word RAM
Howgrave-Graham–Joux [14]	$2^{0.337n}$	$2^{0.294n}$	avg
Becker–Coron–Joux [15]	$2^{0.291n}$		avg
Bonnetain et al. [16]	$2^{0.283n}$		avg
Esser–May [23]	$2^{0.65n}$	polynomial	avg
Either-or subset sum [30]	$2^{0.461n}$		WC, relative
ESS, Yamano–Shibuya [18]	1.6994^n	1.5664^n	WC, relative (preprint)
ESS, Ye [19]	$(5/3)^n$	$(5/3)^n$	WC, relative (preprint)
PESS, Jin–Williams–Zhang [29]	$2^{n/3}$		promise
Quantum, Chukhin et al. [32]	$2^{2n/7}$		WC, quantum

The only statement that would rule out a $2^{n/3}$ algorithm is the *meet-in-the-middle conjecture*, which asserts that $2^{(1/2-\epsilon)n}$ is impossible. It is not implied by any standard hypothesis, and the recent results of Section 6, especially either-or subset sum, have weakened the case for it: a natural problem very close to subset sum does beat $2^{n/2}$ in the worst case. So, as far as is known, a $2^{n/3}$ algorithm for subset sum is possible.

10 Comparison and Common Themes

10.1 The results side by side

Table 8 collects the main results discussed above, with their setting. Reading down the "Setting" column shows the pattern of this survey: every bound below $2^{n/2}$ for subset sum itself is either average-case, quantum, or for a relative with extra structure.

10.2 Theme 1: what to split

All exponential algorithms in this survey split something. Meet-in-the-middle and Schroeppel–Shamir split the *positions* into fixed groups. This is safe for every input, but it gives each solution only one way to be found. The representation technique splits the *solution* itself and gets many ways to find it, which is what allows the filter to cut the search. The gain depends on how many ways there are, which is why richer coefficient sets ($\{-1, 0, 1\}$ and beyond) give better exponents.

10.3 Theme 2: many solutions help

Many of the faster algorithms gain from having many solutions. Wagner's algorithm and the dense algorithms need them. The large-d case of PESS samples from them. ESS and either-or subset

Table 9: Selected milestones.

Year	Milestone
1957	Bellman's dynamic program
1974	Horowitz–Sahni meet-in-the-middle, $2^{n/2}$
1981	Schroeppel–Shamir, memory $2^{n/4}$
1985	Lagarias–Odlyzko lattice attack on low-density instances
1994	Papadimitriou introduces PESS and the class PPP
2002	Wagner's generalised birthday algorithm
2010	Howgrave-Graham–Joux representation technique, $2^{0.337n}$ average case
2011	Becker–Coron–Joux, $2^{0.291n}$ average case
2017	Bringmann, $\tilde{\mathcal{O}}(n+t)$; Bansal et al., polynomial memory
2019	Mucha et al., ESS below $3^{n/2}$
2020	Bonnetain et al., $2^{0.283n}$ average case
2021	Nederlof–Węgrzycki, memory below $2^{n/4}$
2023	Chen et al., $2^{n/2}/n^{0.5023}$ worst case
2024	Jin–Wu, PESS in $2^{0.4n}$
2025	Jin–Williams–Zhang, PESS in $2^{n/3}$; Randolph–Węgrzycki, worst-case balancing
2026	Yamano–Shibuya, ESS in 1.6994^n; Chukhin et al., quantum $2^{2n/7}$

sum benefit from the fact that collisions among subset sums are common. The hard case for subset sum is the opposite: a single solution, and too few collisions among the subset sums to be useful.

10.4 Theme 3: structure or randomness

The recent worst-case results share one pattern. Either the input behaves like a random one in the relevant sense, and the random-case method applies, or it does not, and the failure reveals structure that another method can use. Jin and Wu's PESS algorithm (few non-sums force near powers of two) and the work of Randolph and Węgrzycki (poor mixing forces additive structure) both follow it. For subset sum, the open problem can be phrased in this language: find a dichotomy whose two sides both beat $2^{n/2}$ when the coefficient set is $\{0,1\}$.

10.5 Theme 4: where hardness lives

Taken together, the regimes in this survey narrow down the hard inputs. They have numbers of about n bits, so neither lattices nor dynamic programming apply. Their subset sums cannot all be distinct, because either-or subset sum handles that case below $2^{n/2}$. But collisions cannot be so plentiful that sampling or dense structure theorems find them easily. And because their coefficient set is $\{0,1\}$, coefficient shifting does not apply. Any $2^{(1/2-\epsilon)n}$ algorithm for subset sum has to handle this middle ground: inputs with some, but not many, colliding subset sums.

10.6 Chronology

Table 9 lists the milestones in order. Progress has come in bursts: the classical algorithms of the 1970s and 1980s, the representation technique around 2010, and a cluster of worst-case results for relatives since 2019.

11 Open Problems

1. Subset sum below $2^{n/2}$. Is there an algorithm that solves subset sum in $\mathcal{O}^*(2^{(1/2-\epsilon)n})$ time on every input, for some $\epsilon > 0$? This is the central question, and $2^{n/3}$ would be a natural target since it matches PESS. Partition and balancing over $[\pm 2]$ are open in the same sense.

2. A source of representations for $\{0, 1\}$. The worst-case representation technique needs many representations of a solution. For $\{0, 1\}$ these cannot come from coefficient shifting. Can they come from somewhere else, for example from random restrictions of the input, from a change of target, or from combining several related instances?

3. Closing the gap between random and worst case. On random inputs subset sum takes about $2^{0.283n}$ steps. Is there a worst-case algorithm that matches this up to a small loss, perhaps via a dichotomy of the kind used for PESS and either-or subset sum? Even a worst-case bound of $2^{0.49n}$ would be a breakthrough.

4. The meet-in-the-middle conjecture. Can the conjecture be derived from a standard hypothesis, or refuted? Either answer would settle the status of the central question. A conditional lower bound that matches $2^{n/2}$ would explain fifty years without progress.

5. Memory. At time $2^{n/2}$, the best memory is about $2^{0.246n}$, and with polynomial memory the best time is $2^{0.86n}$. What is the best time–memory trade-off for every input?

6. Lessons from the relatives. PESS, ESS and either-or subset sum all beat their baselines. Is there a reduction that transfers part of this progress to subset sum, for instance by showing that inputs with many colliding subset sums are easy?

12 Conclusion

For every input, subset sum is still solved in about $2^{n/2}$ steps, as it was fifty years ago. Around that bound the field has moved a lot. Random inputs can be solved in about $2^{0.283n}$ steps, small numbers in nearly linear time in the target, and several close relatives now have worst-case algorithms below their baselines, down to $2^{n/3}$ for PESS. Comparing these results shows what they rely on: many solutions, a promise, extra coefficients, or a random-like input. Plain subset sum, with a possibly unique solution, no promise and the coefficient set $\{0, 1\}$, offers none of these. A worst-case algorithm below $2^{n/2}$ would need a new idea, and no known lower bound rules one out.

References

[1] R. Merkle and M. Hellman, "Hiding information and signatures in trapdoor knapsacks," *IEEE Trans. Inf. Theory* 24(5):525–530, 1978.

[2] C. H. Papadimitriou, "On the complexity of the parity argument and other inefficient proofs of existence," *J. Comput. Syst. Sci.* 48(3):498–532, 1994.

[3] D. Pisinger, "Linear time algorithms for knapsack problems with bounded weights," *J. Algorithms* 33(1):1–14, 1999.

[4] E. Szemerédi and V. Vu, "Finite and infinite arithmetic progressions in sumsets," *Ann. of Math.* 163(1):1–35, 2006.

[5] A. K. Lenstra, H. W. Lenstra, L. Lovász, "Factoring polynomials with rational coefficients," *Math. Ann.* 261:515–534, 1982.

[6] R. Impagliazzo and R. Paturi, "On the complexity of k-SAT," *J. Comput. Syst. Sci.* 62(2):367–375, 2001.

[7] E. Horowitz and S. Sahni, "Computing partitions with applications to the knapsack problem," *J. ACM* 21(2):277–292, 1974.

[8] R. Schroeppel and A. Shamir, "A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems," *SIAM J. Comput.* 10(3):456–464, 1981.

[9] X. Chen, Y. Jin, T. Randolph, R. A. Servedio, "Subset sum in time $2^{n/2}/\mathrm{poly}(n)$," RANDOM 2023; arXiv:2301.07134.

[10] J. Nederlof and K. Węgrzycki, "Improving Schroeppel and Shamir's algorithm for subset sum via orthogonal vectors," STOC 2021; arXiv:2010.08576.

[11] E. Belova, I. Chukhin, A. Kulikov, I. Mihajlin, "Improved space bounds for subset sum," ESA 2024; arXiv:2402.13170.

[12] N. Bansal, S. Garg, J. Nederlof, N. Vyas, "Faster space-efficient algorithms for subset sum, k-sum, and related problems," STOC 2017; *SIAM J. Comput.* 47(5):1755–1777, 2018.

[13] D. Lokshtanov and J. Nederlof, "Saving space by algebraization," STOC 2010.

[14] N. Howgrave-Graham and A. Joux, "New generic algorithms for hard knapsacks," EUROCRYPT 2010; IACR ePrint 2010/189.

[15] A. Becker, J.-S. Coron, A. Joux, "Improved generic algorithms for hard knapsacks," EUROCRYPT 2011; IACR ePrint 2011/474.

[16] X. Bonnetain, R. Bricout, A. Schrottenloher, Y. Shen, "Improved classical and quantum algorithms for subset-sum," ASIACRYPT 2020; ePrint 2020/168.

[17] T. Randolph and K. Węgrzycki, "Beating meet-in-the-middle for subset balancing problems," arXiv:2511.10823, 2025.

[18] Yamano and Shibuya, "Faster exact algorithms for Equal-Subset-Sum," arXiv:2607.09289, 2026.

[19] L. Ye, "Solving Equal Subset Sum in $O^*((5/3)^n)$ time," arXiv:2608.08260, 2026.

[20] C. Jin and H. Wu, "A faster algorithm for Pigeonhole Equal Sums," arXiv:2403.19117, 2024.

[21] C. Jin and H. Wu, "A simple near-linear pseudopolynomial time randomized algorithm for subset sum," SOSA 2019; arXiv:1807.11597.

[22] M. Mucha, J. Nederlof, J. Pawlewicz, K. Węgrzycki, "Equal-subset-sum faster than the meet-in-the-middle," ESA 2019.

[23] A. Esser and A. May, "Low weight discrete logarithm and subset sum in $2^{0.65n}$ with polynomial memory," EUROCRYPT 2020.

[24] K. Bringmann and V. Nakos, "Top-k-convolution and the quest for near-linear output-sensitive subset sum," STOC 2020.

[25] V. Lyubashevsky, "The parity problem in the presence of noise, decoding random linear codes, and the subset sum problem," RANDOM 2005.

[26] A. Shallue, "An improved multi-set algorithm for the dense subset sum problem," ANTS 2008.

[27] M. J. Coster, A. Joux, B. A. LaMacchia, A. M. Odlyzko, C.-P. Schnorr, J. Stern, "Improved low-density subset sum algorithms," *Comput. Complexity* 2(2):111–128, 1992.

[28] A. Abboud, K. Bringmann, D. Hermelin, D. Shabtay, "SETH-based lower bounds for subset sum and bicriteria path," SODA 2019.

[29] C. Jin, R. Williams, S. Zhang, "New algorithms for Pigeonhole Equal Subset Sum," ESA 2025, LIPIcs 351:86.

[30] T. Randolph, "Exact and parameterized algorithms for subset sum problems," PhD thesis, Columbia University, 2024.

[31] D. Bernstein, S. Jeffery, T. Lange, A. Meurer, "Quantum algorithms for the subset-sum problem," PQCrypto 2013.

[32] I. Chukhin, A. Kulikov, V. Levitskii, I. Mihajlin, "Improved quantum algorithms for subset sum and k-SUM," arXiv:2608.07309, 2026.

[33] D. Wagner, "A generalized birthday problem," CRYPTO 2002.

[34] J. Lagarias and A. Odlyzko, "Solving low-density subset sum problems," J. ACM 32(1):229–246, 1985.

[35] K. Koiliaris and C. Xu, "A faster pseudopolynomial time algorithm for subset sum," SODA 2017.

[36] K. Bringmann, "A near-linear pseudopolynomial time algorithm for subset sum," SODA 2017.

[37] L. Chen, J. Lian, Y. Mao, G. Zhang, "An improved pseudopolynomial time algorithm for subset sum," FOCS 2024.

[38] K. Bringmann, N. Fischer, V. Nakos, "Beating Bellman's algorithm for subset sum," SODA 2025; arXiv:2410.21942.

[39] K. Bringmann and P. Wellnitz, "On near-linear-time algorithms for dense subset sum," SODA 2021; arXiv:2010.09096.

[40] L. Chen, Y. Mao, G. Zhang, "Long arithmetic progressions in sumsets and subset sums," STOC 2025; arXiv:2503.19299.